오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

JavaScript 심화

DAY 19

고급 함수와 고차 함수

변수가 보이는 범위와 클로저, this의 규칙을 보고,
함수를 주고받는 고차 함수로 데이터를 다룹니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **스코프와 클로저를 설명할 수 있다**

   변수가 보이는 범위를 알고, 만들어진 곳의 변수를 기억하는 클로저로 카운터를 만듭니다.

2. **this를 읽고 Math · Date를 쓸 수 있다**

   메서드 · 콜백 · 화살표 함수의 this를 알고, 기본 객체 Math · Date로 계산하고 날짜를 읽습니다.

3. **반복문을 고차 함수로 바꿔 쓸 수 있다**

   함수를 주고받는 고차 함수를 알고, map · filter · reduce로 통계를 냅니다.

오늘의 결과물

두 관의 인원을 따로 세는 counter.js와, 반복문을 메서드로 바꾼 운동 기록 통계 stats.js

ROADMAP

오늘의 흐름

01

스코프

전역 · 함수 · 블록
렉시컬 스코프 · 환경

02

클로저

함수가 기억하는 변수
카운터 · 실습 01

실습 포함

03

this와 기본 객체

부른 모양이 정하는 this
Math · Date

04

고차 함수

함수를 주고받는 함수
reduce 심화 · 실습 02

실습 포함

앞의 두 섹션은 함수가 변수를 찾는 규칙(스코프)과, 그 규칙으로 변수를 기억하는 클로저입니다.
뒤의 두 섹션은 함수 안의 this와 기본 객체 Math · Date, 그리고 함수를 값처럼 주고받는 고차 함수입니다.

되짚기 · 01

var · let · const가 보이는 범위

배운 곳	그때 본 것	오늘
DAY 10 32장	var로 만든 변수는 { } 밖에서도 보임	블록 스코프 — let · const만
DAY 10 35장	보이는 범위 — var는 함수, let · const는 블록 { }	스코프의 세 종류
DAY 12 39장	함수 안의 result는 밖에서 ReferenceError	함수 스코프 · 스코프 체인

오늘은 '어디서 보이는가'에 이름을 붙입니다 — 스코프

그리고 함수가 끝난 뒤에도 그 변수를 기억하는 클로저로 이어집니다.

되짚기 · 02

지금까지 콜백 함수를 넘긴 곳

배운 곳	함수를 넘긴 코드	넘긴 함수를 부르는 쪽
DAY 14 8장	`fruits.forEach((fruit) => console.log(fruit));`	forEach — 값 하나마다
DAY 16 9장	`countBtn.addEventListener("click", addOne);`	브라우저 — 누를 때마다
DAY 17 18장	`setTimeout(sayHello, 2000);`	브라우저 — 2초 뒤에
DAY 18 26장	`.then((response) => response.json())`	Promise — 이행되면

함수를 인수로 받거나 함수를 돌려주는 함수 — 고차 함수

forEach · addEventListener · setTimeout · then이 모두 고차 함수입니다. 오늘은 직접 만들어 봅니다.

준 비

오늘의 셋팅

```
Day19/                          ← 같은 저장소 안에 만듭니다
  index.html                    ← 미리 받은 파일 (장표 예제)
  index.js                      ← 장표를 보며 함께 칩니다
  style.css                     ← 모두 함께 씁니다
  counter.html · counter.js     ← 실습 01
  stats.html · stats.js         ← 실습 02
  stats_loop.js                 ← 반복문으로 쓴 원래 코드 (나눠 줌)
$ git status
/* 오늘도 main에 바로 커밋합니다 */
```

Console을 먼저

오늘 예제는 거의 모두
Console에 찍히는 값으로
확인합니다.

예측하고 실행

실행하기 전에 찍힐 값을
먼저 적어 보고
실제 결과와 맞춰 봅니다.

01

스코프와 렉시컬 환경

스코프 세 종류 · 함수 스코프 · 블록 스코프 · 스코프 체인 · 같은 이름 · 렉시컬 스코프

스코프 — 변수가 보이는 범위

```
const city = "서울";                          전역 스코프

function outer() {                            함수 스코프
  const shop = "1호점";

    if (true) {                               블록 스코프
      const menu = "라떼";
      console.log(city, shop, menu);
    }

  // 여기서 menu는 보이지 않음
}
```

전역 스코프

맨 바깥 — 파일 어디서나 보입니다.

함수 스코프

함수 { } 안에서 만든 것은 그 함수 안에서만.

블록 스코프

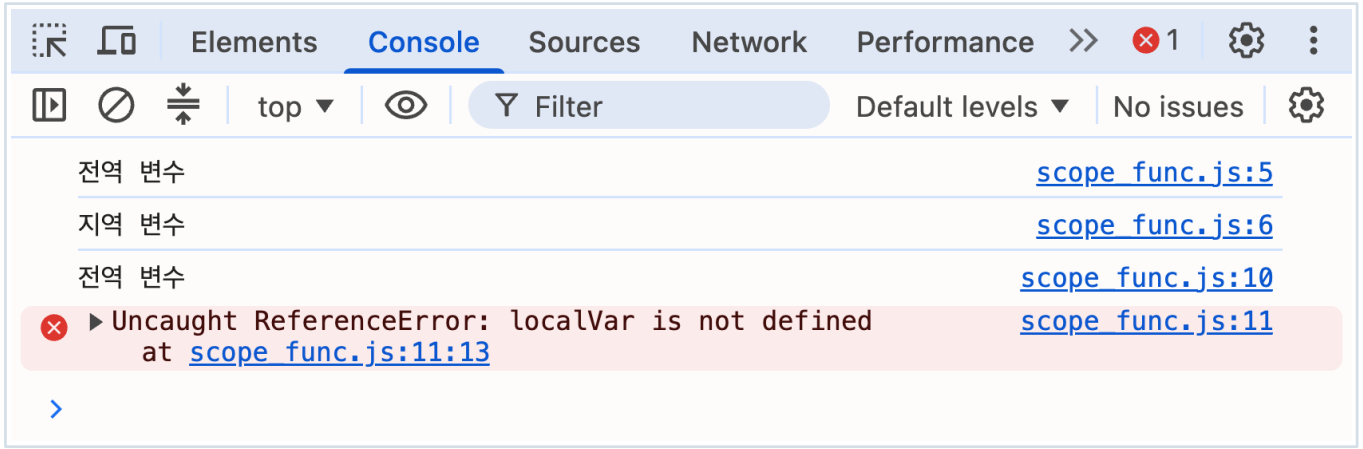
let · const는 if · for의 { } 안에서만.

함수 안에서 만든 변수는 함수 안에서만 사용

```
let globalVar = "전역 변수";

function myFunction() {
  let localVar = "지역 변수";
  console.log(globalVar);
  console.log(localVar);
}

myFunction();
console.log(globalVar);
console.log(localVar);
```



함수 스코프

localVar — 함수 안에서만
밖에서 부르면 ReferenceError
globalVar — 어디서나

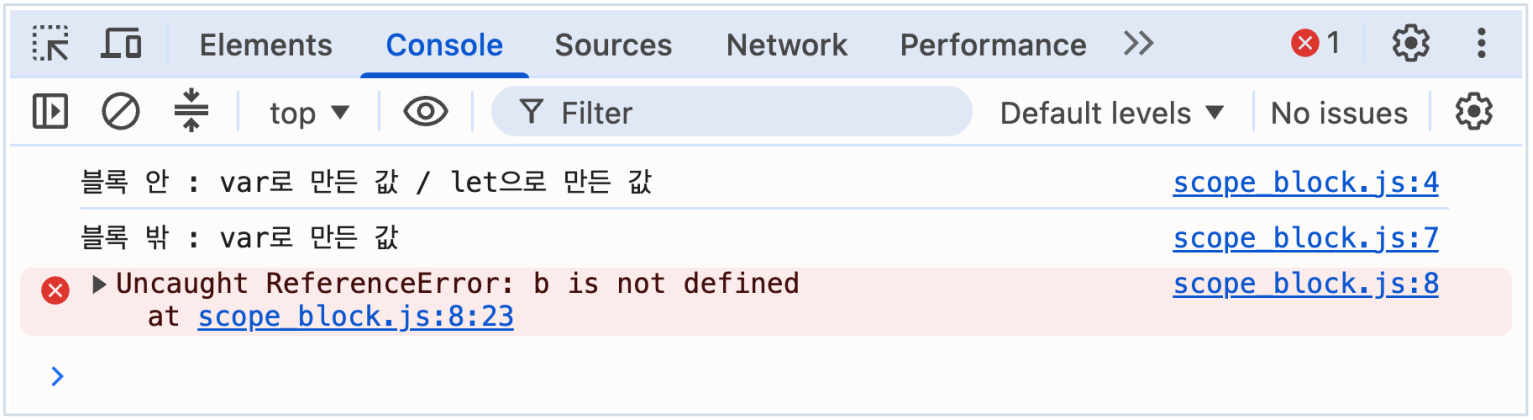
let · const는 { } 안에서만 — 블록 스코프

```
if (true) {
  var a = "var로 만든 값";
  let b = "let으로 만든 값";
  console.log("블록 안 :", a, "/", b);
}

console.log("블록 밖 :", a);
console.log("블록 밖 :", b);
```

블록 스코프

let · const — { } 안에서만
var — { } 밖에서도 보임
그래서 var는 새로 쓰지 않습니다



안에서는 밖이 보이고, 밖에서는 안이 안 보임

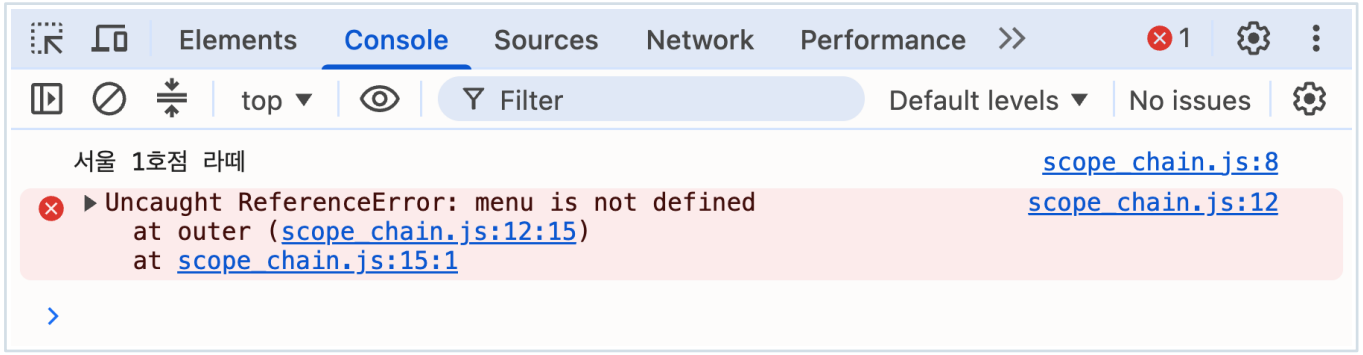
```
const city = "서울";

function outer() {
  const shop = "1호점";

  function inner() {
    const menu = "라떼";
    console.log(city, shop, menu);
  }

  inner();
  console.log(menu);
}

outer();
```



안에서 밖으로 찾음

inner — city · shop · menu 모두

outer — inner 안의 menu는 못 봄

끝까지 없으면 ReferenceError

같은 이름이면 가까운 스코프 우선

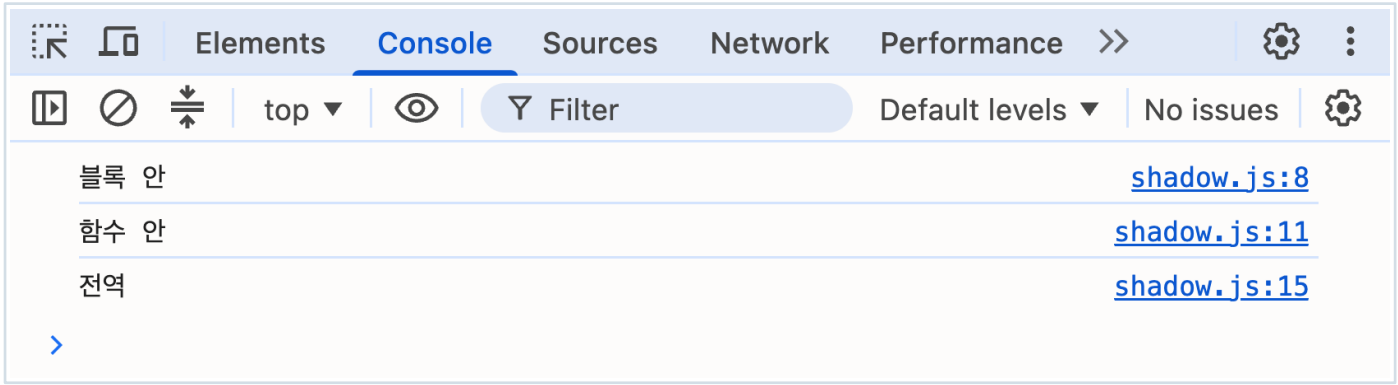
```
const message = "전역";

function greet() {
  const message = "함수 안";

  if (true) {
    const message = "블록 안";
    console.log(message);
  }

  console.log(message);
}

greet();
console.log(message);
```



같은 이름이면

지금 스코프에서 먼저 찾음
없을 때만 바깥으로
바깥 변수는 바뀌지 않음

어디서 불렀나가 아니라, 어디서 만들었나

```
const where = "전역";

function show() {
  console.log(where);
}

function run() {
  const where = "run 안";
  show();
}

run();
```



렉시컬 스코프

바깥 스코프는 만든 곳이 정함
show는 전역에서 만듦
run 안에서 불러도「전역」

렉시컬 환경 — 이름과 값, 바깥으로 가는 연결

전역 — 파일을 읽을 때

`where: "전역"`

`show: 함수` `run: 함수`

↑ 바깥

run을 부를 때

`where: "run 안"`

바깥 → 전역

↑ 바깥

show를 부를 때

(만든 변수 없음)

바깥 → 전역 (만든 곳)

렉시컬 환경

스코프마다 만들어지는 기록
— 변수 이름과 값
— 바깥 환경으로 가는 연결

바깥 연결은 함수를
만든 곳으로 정해집니다.

show는 run 안에서 불렸지만, 바깥 연결은 show를 만든 전역입니다

여기까지 정리

1	**스코프**	변수를 쓸 수 있는(보이는) 범위
2	**함수 스코프**	함수 안에서 만든 변수는 그 함수 안에서만
3	**블록 스코프**	let · const로 만든 변수가 { } 안에서만 보이는 범위
4	**스코프 체인**	안에서 밖으로 한 겹씩 찾음 — 끝까지 없으면 ReferenceError
5	**렉시컬 스코프**	어디서 불렀는지가 아니라 어디에 만들었는지로 바깥이 정해짐

다음은 클로저 — 함수가 끝난 뒤에도 바깥 변수를 기억합니다

02

클로저

클로저란 · 끝난 함수의 변수 · 카운터 세 가지 · 즉시 실행 함수 · 두 카운터 · 반복문 · 실습 01

클로저 — 함수와, 함수가 만들어진 곳의 변수

MDN

함수 + 렉시컬 환경

함수와, 함수를 둘러싼 상태를
함께 묶은 것
함수를 만들 때마다 생김

다르게 말하면

함수 + 선언된 곳

클로저는 함수와 그 함수가
선언된 렉시컬 환경과의
조합입니다.

함수가 끝나도, 그 함수의 변수를 안쪽 함수가 기억합니다

다음 장부터 Console에서 실제로 확인합니다.

함수가 끝난 뒤에도 기억하는 변수

```
const x = 1;

function outer() {
  const x = 10;
  const inner = function () {
    console.log(x);
  };
  return inner;
}

const innerFunc = outer();
innerFunc();
```



클로저

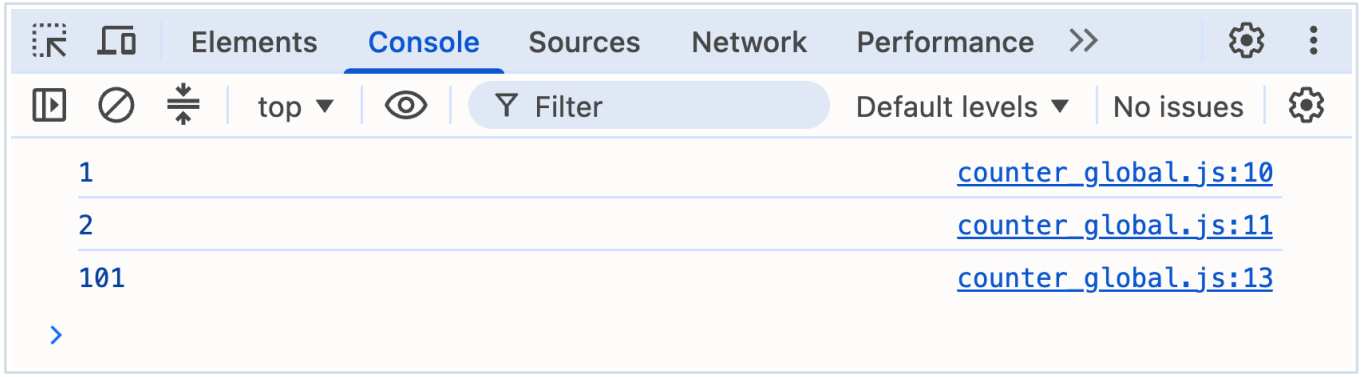
outer는 이미 끝났지만
inner가 outer의 x = 10을 기억
전역 x = 1이 아님

카운터 ① — 누구나 바꾸는 전역 변수

```
// 카운트 상태 변수
let num = 0;

// 카운트 상태 변경 함수
const increase = function () {
  // 카운트 상태를 1만큼 증가시킨다.
  return ++num;
};

console.log(increase());
console.log(increase());
num = 100; // 누구나 바꿀 수 있습니다
console.log(increase());
```



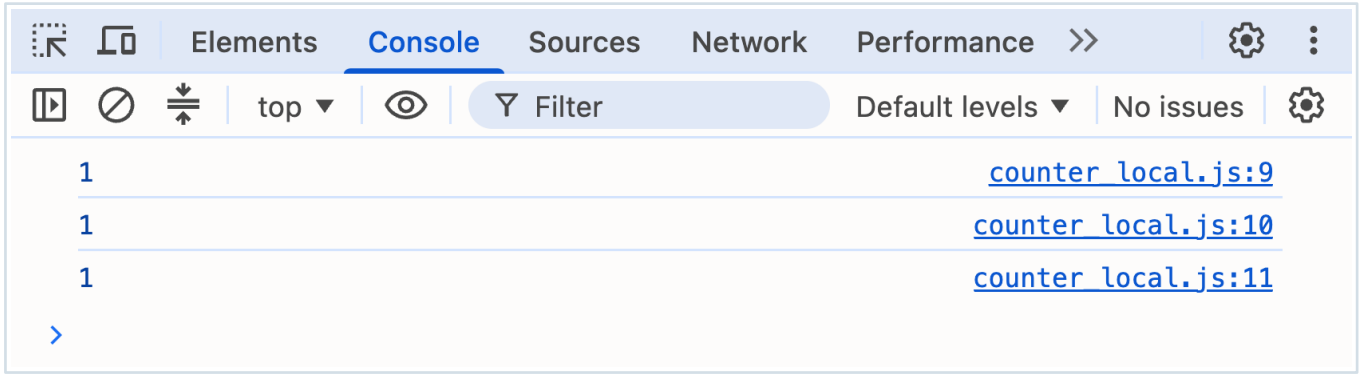
전역 변수 카운터

1, 2 — 잘 셉니다
num = 100 한 줄에 101
누구나 바꿀 수 있음

카운터 ② — 함수 안에 두면 늘 1

```
// 카운트 상태 변경 함수
const increase = function () {
  // 카운트 상태 변수
  let num = 0;
  // 카운트 상태를 1만큼 증가시킨다.
  return ++num;
};

console.log(increase());
console.log(increase());
console.log(increase());
```



지역 변수 카운터

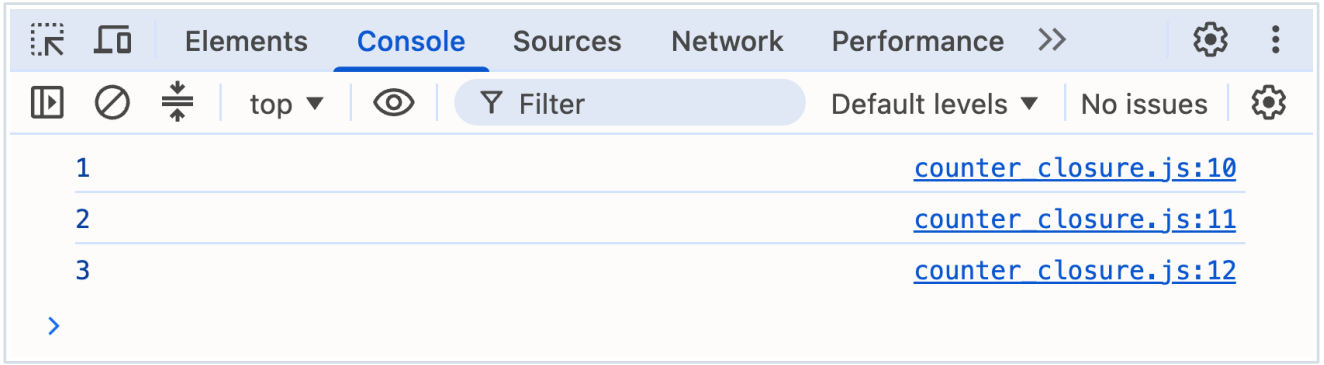
부를 때마다 num = 0부터
그래서 늘 1
앞의 값을 기억하지 못함

카운터 ③ — 클로저로 숨기고 기억하기

```
function makeIncrease() {
  let num = 0; // 이 함수 안에만 있는 변수

  return function () {
    return ++num;
  };
}

const increase = makeIncrease();
console.log(increase());
console.log(increase());
console.log(increase());
```



클로저 카운터

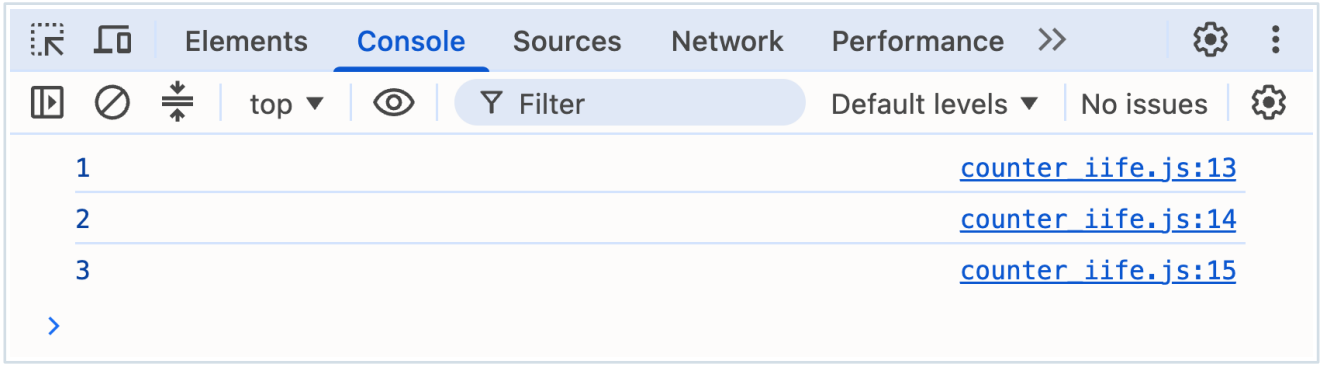
num은 makeIncrease 안에
돌려준 함수가 num을 기억
1, 2, 3 — 밖에서는 못 바꿈

즉시 실행 함수로 만든 카운터

```
// 카운트 상태 변경 함수
const increase = (function () {
  // 카운트 상태 변수
  let num = 0;

  // 클로저
  return function () {
    // 카운트 상태를 1만큼 증가시킨다.
    return ++num;
  };
}());

console.log(increase());
console.log(increase());
console.log(increase());
```



즉시 실행 함수

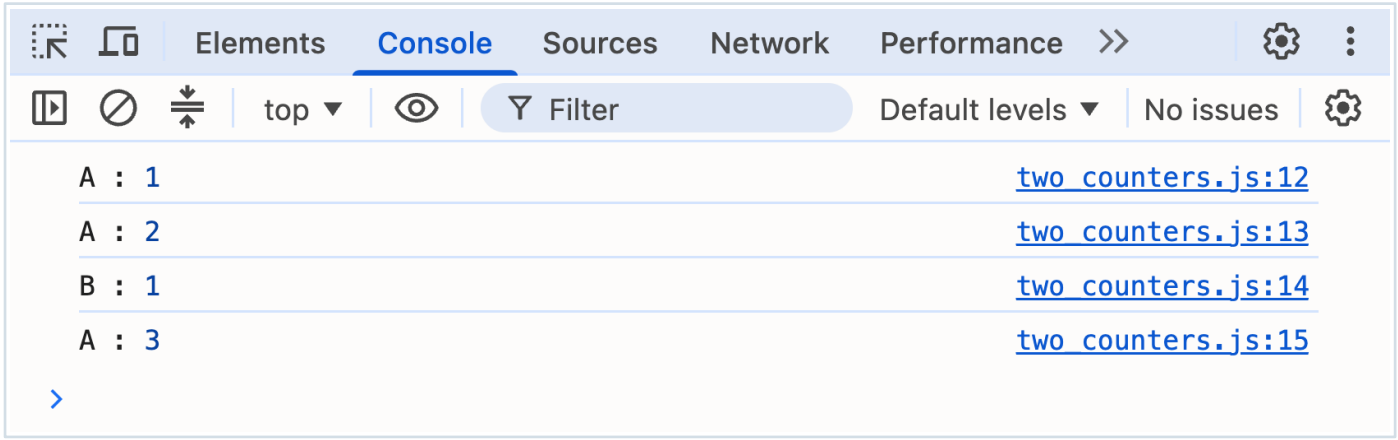
만들자마자 바로 실행 — ( )
돌려준 함수가 increase에
결과는 앞 장과 같음

만들 때마다 따로 기억

```
function makeCounter() {
  let count = 0;
  return function () {
    count += 1;
    return count;
  };
}

const counterA = makeCounter();
const counterB = makeCounter();

console.log("A :", counterA());
console.log("A :", counterA());
console.log("B :", counterB());
console.log("A :", counterA());
```



부를 때마다 새 클로저

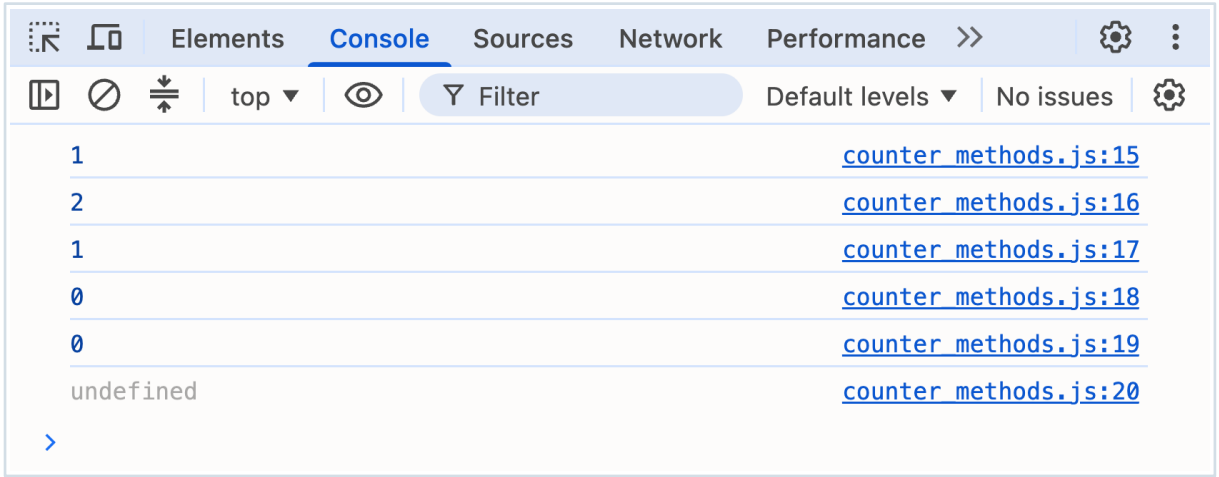
makeCounter()마다 count가 따로
A를 세도 B는 그대로
A : 1, 2 · B : 1 · A : 3

메서드 여러 개가 함께 쓰는 숨긴 변수 하나

```
const counter = (function () {
  let num = 0; // 카운트 상태 변수

  // 클로저인 메서드를 갖는 객체를 반환한다.
  return {
    increase() {
      return ++num;
    },
    decrease() {
      return num > 0 ? --num : 0;
    },
  };
}());

console.log(counter.increase());
console.log(counter.increase());
console.log(counter.decrease());
console.log(counter.decrease());
console.log(counter.decrease());
console.log(counter.num);
```



카운트 증가 · 감소

increase · decrease가 num을 공유
0 밑으로는 내려가지 않음
counter.num — undefined

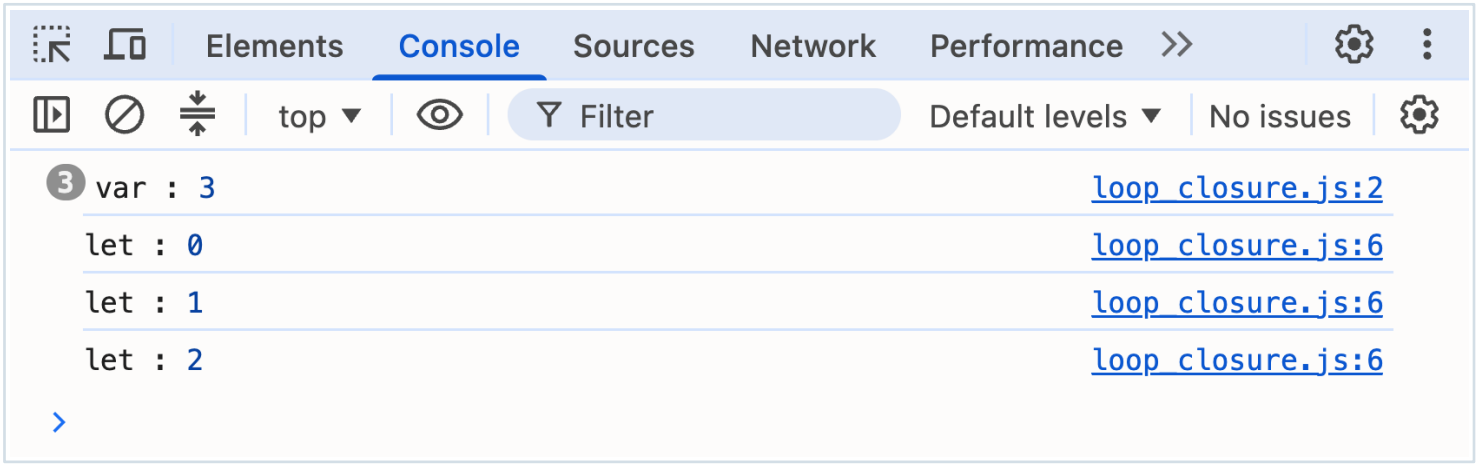
반복문 안의 클로저 — var와 let

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log("var :", i), 0);
}

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log("let :", j), 0);
}
```

i 하나 vs 반복마다 새 j

var i — 하나뿐, 세 함수가 함께 봄
다 돈 뒤의 i는 3
let j — 반복마다 새로 만들어짐



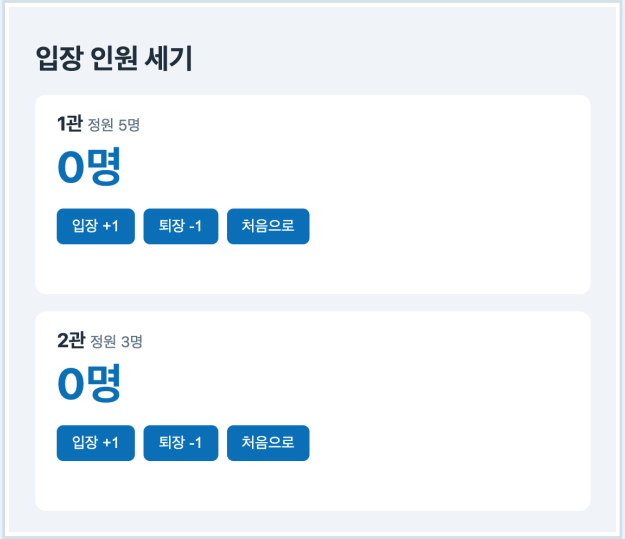
실습 01

입장 인원 세기

counter.html은 그대로 두고 counter.js만 써서, 1관 · 2관의 인원을 정원까지 따로 셉니다.
클로저로 count를 숨기고, 버튼은 DAY 16의 click으로 연결합니다.

만들 것

① createCounter — count를 숨긴 카운터
increase · decrease · reset · getCount
② connectHall — 관 하나를 화면과 연결
③ show — 인원과「정원이 찼습니다」
④ 버튼 셋에 click 연결
⑤ 1관(정원 5) · 2관(정원 3) 만들기



처음 열면 — 두 관 모두 0명

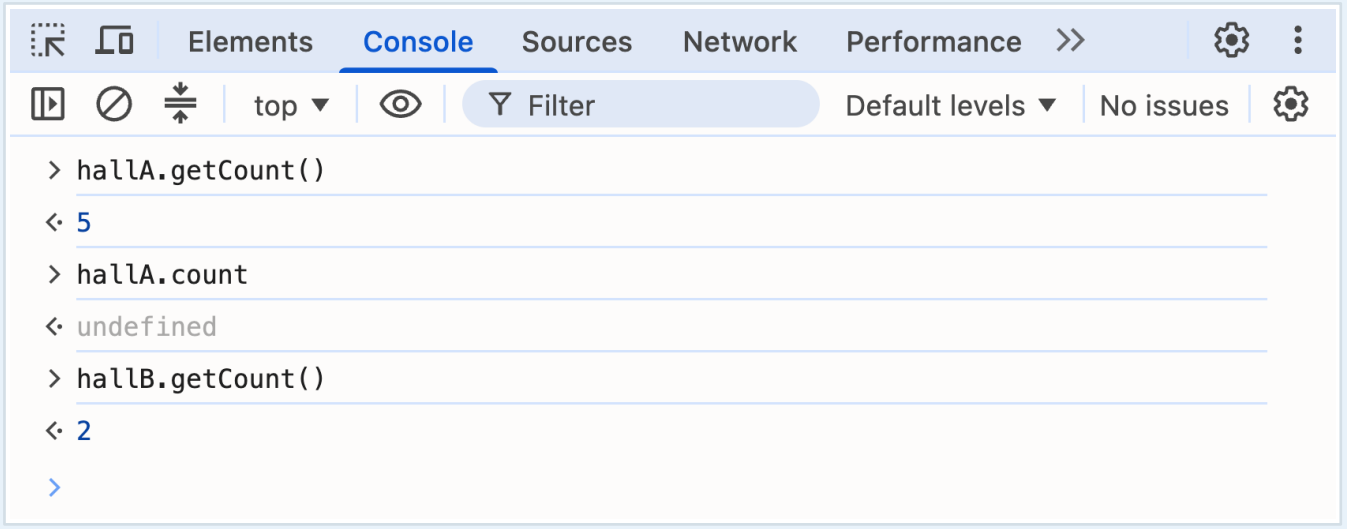


1관 다섯 번 · 2관 두 번 누른 뒤

실습 01

메서드로만 보이는 숨긴 count

다섯 번 · 두 번 누른 뒤 Console에 hallA.getCount(), hallA.count, hallB.getCount()를 쳐 봅니다.
count는 createCounter 안에 숨어 있어, 돌려준 메서드로만 읽고 바꿀 수 있습니다.



Console에 직접 친 세 줄 — 줄마다 아래가 돌려받은 값

결과 읽기

hallA.getCount() — 5
hallA.count — undefined
hallB.getCount() — 2

두 관은 따로 세고,
count는 메서드로만 보입니다.

실습 01 · 확인

두 관이 함께 움직이면 — 변수 위치부터 확인

바꿀 곳	결과
① 5번 줄 let count = 0;을 2번 줄(빈 줄)로 옮기기	두 관이 count 하나를 나눠 씀 — 1관 2명 뒤 2관을 누르면 3명
② 38번 줄의 counter.getCount()를 counter.count로	두 관 모두 「undefined명」 — 눌러도 그대로
③ 9번 줄의 count < max를 count <= max로	1관이 6명까지 — 6명이면 안내 문구가 사라짐
④ 57번 줄 show(); 앞에 // 붙이기	처음에 인원 칸이 비어 있음 — 누르면 나옴

Console이 조용해도 — 숫자가 이상하면 count가 어느 스코프에 있는지 봅니다

여기까지 정리

1	**클로저**	함수와, 그 함수가 만들어진 곳의 변수(렉시컬 환경)를 함께 기억하는 것
2	**데이터 은닉**	변수를 함수 안에 숨기고, 돌려준 함수(메서드)로만 바꿈
3	**즉시 실행 함수**	만들자마자 바로 실행하는 함수 — (function () { … })()
4	**따로 기억**	만드는 함수를 부를 때마다 새 변수와 새 클로저
5	**반복문**	var는 i 하나를 함께 보고, let은 반복마다 새로 만들어짐

다음은 this — 함수 안의 this는 부르는 모양이 정합니다

03

this와 기본 객체

this란 · 떼어 내 부르면 · 콜백과 this · 화살표 함수 · 이벤트 리스너 · Math · Date

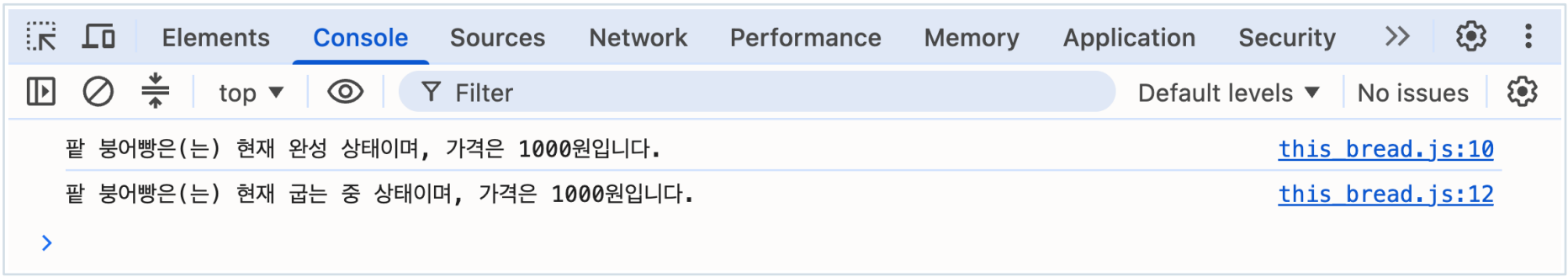
this — 메서드를 부른 객체

```
const bread = {
  name: "팥 붕어빵",
  price: 1000,
  bakeryStatus: "완성",
  describe() {
    return `${this.name}은(는) 현재 ${this.bakeryStatus} 상태이며, 가격은 ${this.price}원입니다.`;
  },
};

console.log(bread.describe());
bread.bakeryStatus = "굽는 중";
console.log(bread.describe());
```

bread.describe()

this는 bread — 부른 객체
this.name · this.price
바꾼 값도 그대로 읽음

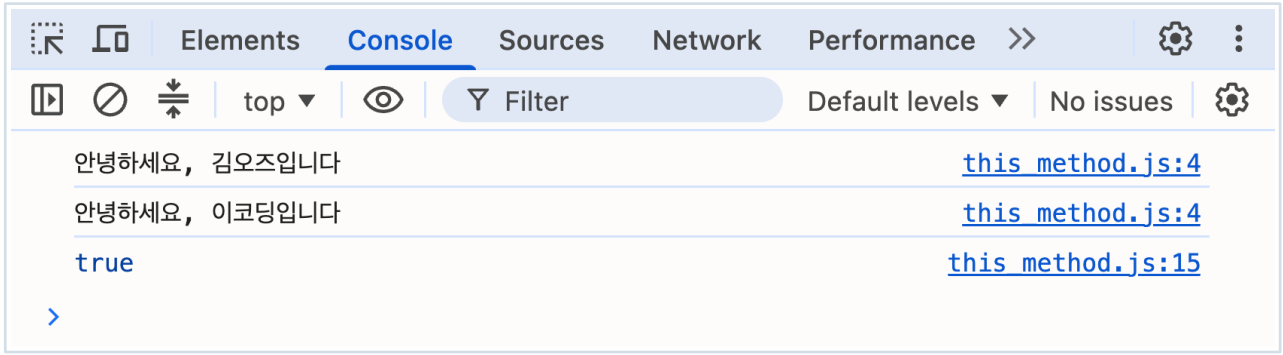


같은 함수, 부른 객체가 다르면 this도 다름

```
const shop = {
  owner: "김오즈",
  hello() {
    console.log(`안녕하세요, ${this.owner}입니다`);
  },
};

const shop2 = {
  owner: "이코딩",
  hello: shop.hello, // 같은 함수를 담음
};

shop.hello();
shop2.hello();
console.log(shop.hello === shop2.hello);
```



obj.method()

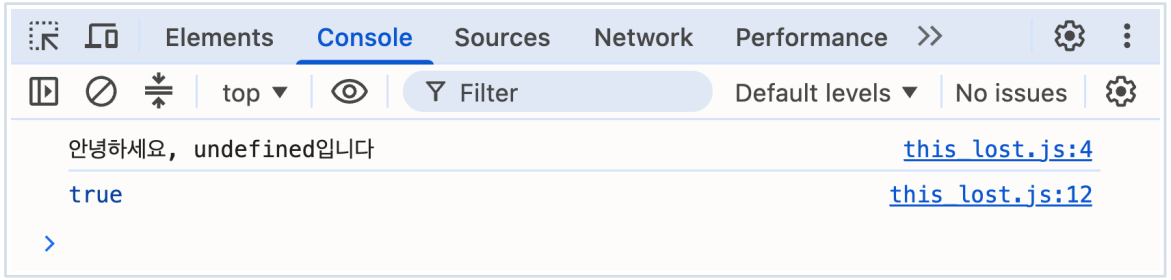
shop.hello() — this는 shop
shop2.hello() — this는 shop2
같은 함수, 부른 객체가 다름

떼어 내 부르면 잃는 this

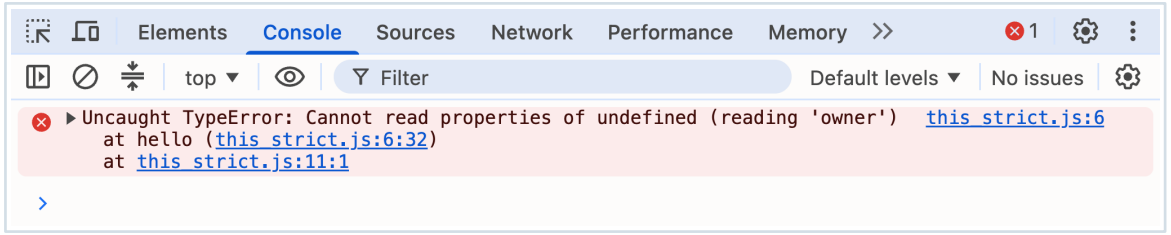
```
const shop = {
  owner: "김오즈",
  hello() {
    console.log(`안녕하세요, ${this.owner}입니다`);
  },
};

const hello = shop.hello; // 메서드를 떼어 변수에 담음
hello();

function whoIsThis() {
  console.log(this === window);
}
whoIsThis();
```



Console — this는 window, owner는 없음



맨 위에 "use strict"; — this가 undefined라 TypeError

콜백으로 넘겨도 잃는 this

```
const shop = {
  owner: "김오즈",
  hello() {
    console.log(`안녕하세요, ${this.owner}입니다`);
  },
};

setTimeout(shop.hello, 1000); // ① 메서드만 넘김
setTimeout(() => shop.hello(), 1000); // ② 화살표 함수 안에서 부름
```



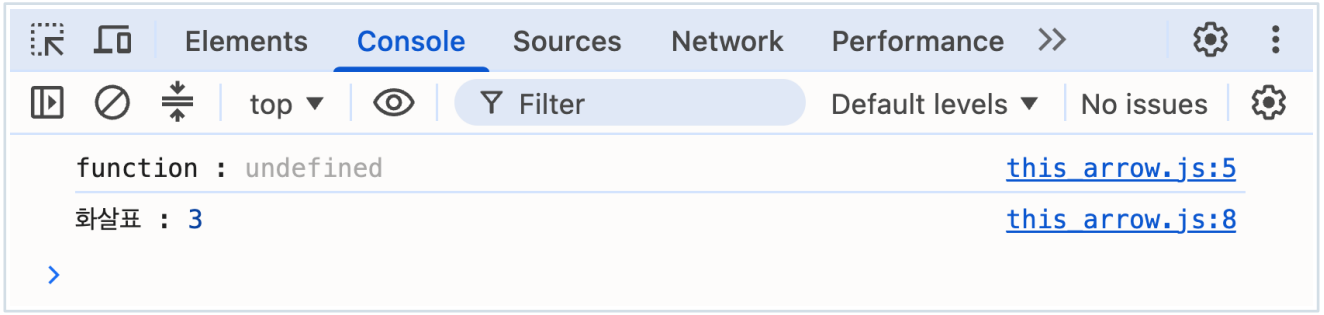
setTimeout에 넘길 때

① 함수만 넘어가 this는 window
② 화살표 안에서 shop.hello()

this를 따로 만들지 않는 화살표 함수

```
const timer = {
  seconds: 3,
  start() {
    setTimeout(function () {
      console.log("function :", this.seconds);
    }, 0);
    setTimeout(() => {
      console.log("화살표 :", this.seconds);
    }, 0);
  },
};

timer.start();
```



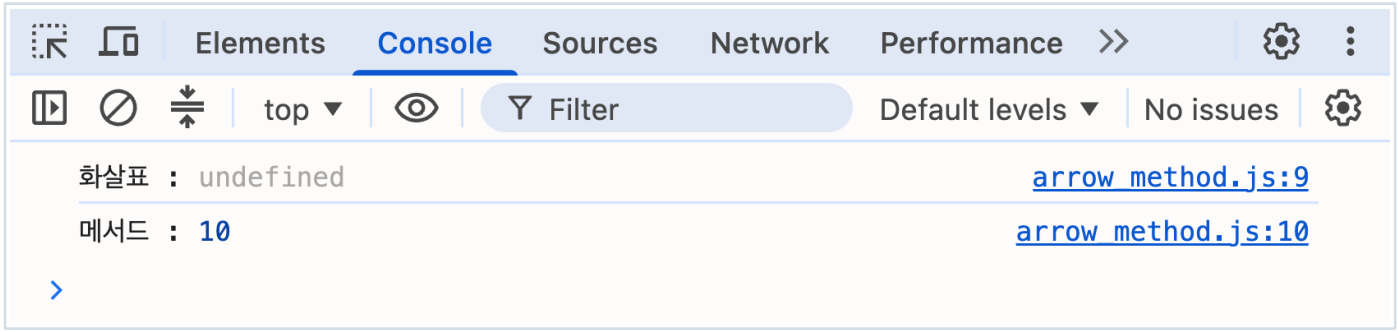
바깥의 this를 그대로

function — setTimeout이 부름 → window
화살표 — start의 this(timer)
그래서 seconds는 3

메서드는 화살표 함수로 만들지 않기

```
const box = {
  size: 10,
  getSize: () => this.size,
  getSize2() {
    return this.size;
  },
};

console.log("화살표 :", box.getSize());
console.log("메서드 :", box.getSize2());
```



메서드는 메서드 모양으로

getSize — this는 바깥(window)
getSize2() — this는 box
메서드에는 화살표를 쓰지 않음

이벤트 리스너의 this — 리스너를 단 요소

```
const likeBtn = document.querySelector("#like-btn");

likeBtn.addEventListener("click", function () {
  this.textContent = "눌렀어요";
  console.log("function :", this);
});

likeBtn.addEventListener("click", () => {
  console.log("화살표 :", this === window);
});
```

function이면 this는 버튼

리스너를 단 요소 — likeBtn
화살표 함수면 바깥의 this(window)



Console — 버튼을 한 번 누른 뒤



화면 — 버튼의 글이 바뀜

기본 객체 Math로 계산하기

```
console.log(Math.round(4.5));          // 5  반올림
console.log(Math.floor(4.7));          // 4  내림
console.log(Math.ceil(4.2));           // 5  올림
console.log(Math.max(72, 95, 58));     // 95  가장 큰 수
console.log(Math.min(72, 95, 58));     // 58  가장 작은 수
console.log(Math.abs(-3));             // 3  절댓값
console.log(Math.PI);                  // 3.141592653589793
```

기본 객체

JavaScript가 미리 만들어 둔 객체
Math는 new 없이 Math.이름(값)
DAY 18의 JSON도 기본 객체

round는 반올림, floor는 내림, ceil은 올림

Math는 계산에 쓰는 함수와 값(Math.PI)을 모아 둔 기본 객체라, 만들지 않고 바로 씁니다(MDN Math).

Math.random으로 무작위 수 만들기

```
console.log(Math.random());    // 0 이상 1 미만, 부를 때마다 다름

const dice = Math.floor(Math.random() * 6) + 1;
console.log(dice);             // 1 ~ 6 가운데 하나

const menus = ["아메리카노", "라떼", "모카"];
const i = Math.floor(Math.random() * menus.length);
console.log(menus[i]);         // 셋 가운데 하나
```

읽는 법

random()은 0 이상 1 미만
× 6 → 0 이상 6 미만
floor → 0 ~ 5, + 1 → 1 ~ 6

Math.floor(Math.random() * 개수)는 0부터 개수 - 1까지

배열의 번호도 0부터 length - 1까지라, 무작위로 하나를 고를 때 그대로 씁니다.

Date로 날짜와 요일 읽기

```
const day = new Date(2026, 9, 29);    // 달은 0부터 — 9가 10월
console.log(day.getFullYear());       // 2026
console.log(day.getMonth() + 1);      // 10
console.log(day.getDate());           // 29
console.log(day.getDay());            // 4   일요일 0 … 목요일 4
console.log(day.toLocaleDateString("ko-KR"));   // 2026. 10. 29.

const now = new Date();               // 지금 이 순간
console.log(now.getHours());          // 지금 시각의 시
```

달은 0부터

getMonth()는 0 ~ 11
보여 줄 때는 + 1
만들 때 10월은 9

new Date(년, 달 - 1, 일)로 만들고 get…으로 읽기

요일 getDay()도 0(일요일)부터 6(토요일)까지의 번호입니다(MDN Date).

여기까지 정리

1	this	메서드 안에서, 그 메서드를 부른 객체를 가리키는 말
2	떼어 내 부르면	객체 없이 부르면 window — 엄격 모드면 undefined
3	화살표 함수	this를 따로 만들지 않고 바깥의 this를 그대로 씀
4	Math	반올림 · 내림 · 올림, 큰 수 · 작은 수, 무작위 수를 돌려주는 기본 객체
5	Date	new Date(년, 달 - 1, 일)로 만들고 get…으로 읽는 날짜 — 달은 0부터

다음은 고차 함수 — 함수를 주고받으며 데이터를 다룹니다

04

고차 함수

고차 함수 · 함수를 돌려주는 함수 · 명령형과 함수형 · 이어 쓰기 · reduce 심화 · 실습 02

고차 함수 — 함수를 받거나 함수를 돌려주는 함수

함수를 받는다

콜백을 받는 함수

forEach · map · filter · reduce
addEventListener · setTimeout · then
넘긴 함수를 그 함수가 부름

함수를 돌려준다

함수를 만들어 주는 함수

makeIncrease · makeCounter
add · makeMultiplier — 다음 장
돌려받은 함수가 바깥 변수를 기억

JavaScript의 함수는 값입니다 — 변수에 담고, 넘기고, 돌려줍니다

MDN은 이런 함수를 일급 함수라 하고, 함수를 주고받는 함수를 고차 함수라고 합니다.

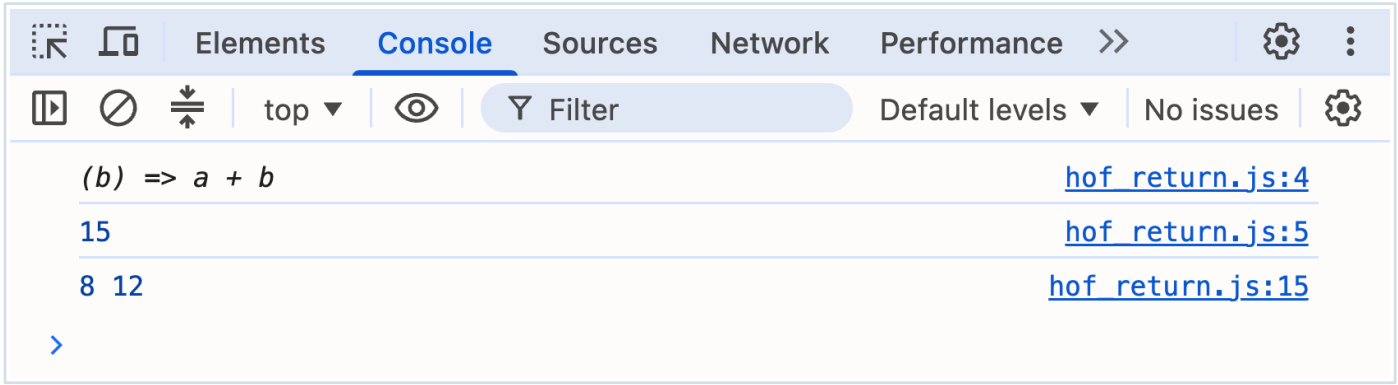
함수를 돌려주는 함수 — 값을 기억한 함수

```
const add = (a) => (b) => a + b;
const add5 = add(5);

console.log(add5);
console.log(add5(10));

function makeMultiplier(n) {
  return function (x) {
    return x * n;
  };
}

const double = makeMultiplier(2);
const triple = makeMultiplier(3);
console.log(double(4), triple(4));
```

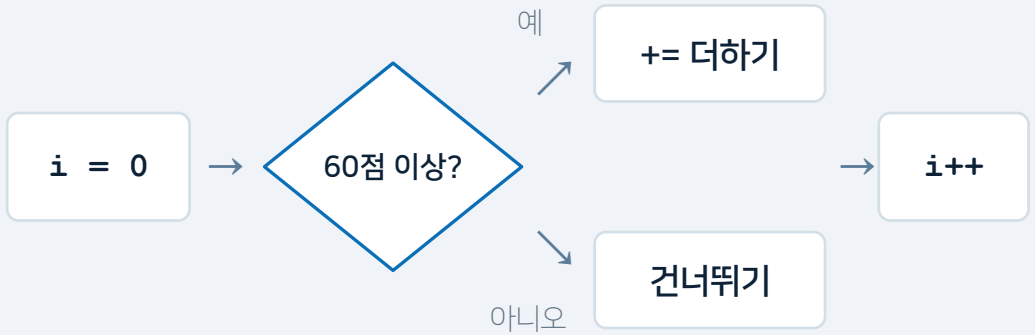


돌려받은 함수

add(5) — a = 5를 기억한 함수
add5(10) — 15
makeMultiplier(2) · (3) — 따로

반복문과 메서드 이어 쓰기 — 다른 흐름

반복문 (명령형)



i++ 다음에는 조건으로 돌아가 다시 — 반복

메서드 이어 쓰기 (함수형)



한 방향으로 — 되돌아가는 길이 없음

반복문은 「어떻게」를 한 단계씩, 메서드 이어 쓰기는 「무엇을」 차례로

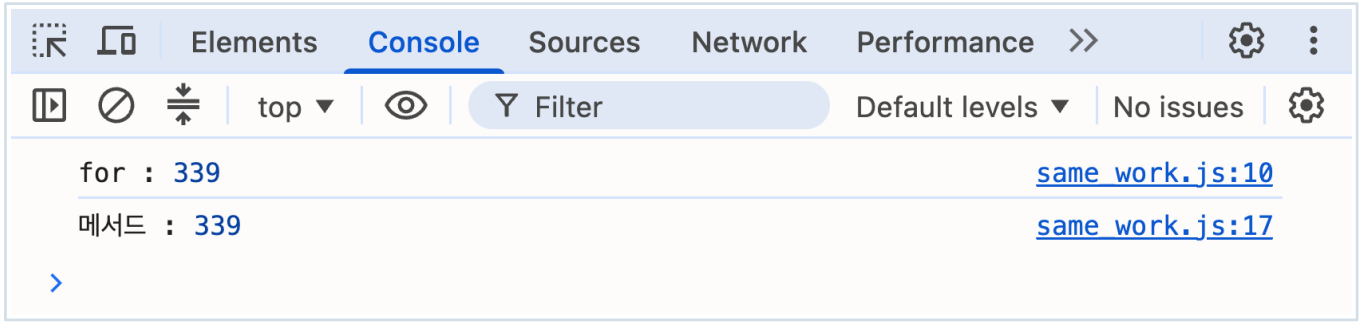
명령형 코드에는 갈림길과 되돌아가는 길이 있고, 함수형 코드는 한 방향으로 흐릅니다.

같은 일, 두 모양 — for와 메서드 이어 쓰기

```
const scores = [72, 95, 58, 88, 64];

// 반복문 — 어떻게 할지를 한 단계씩
let total1 = 0;
for (let i = 0; i < scores.length; i++) {
  if (scores[i] >= 60) {
    total1 += scores[i] + 5;
  }
}
console.log("for :", total1);

// 메서드 이어 쓰기 — 무엇을 할지를 차례로
const total2 = scores
  .filter((score) => score >= 60)
  .map((score) => score + 5)
  .reduce((sum, score) => sum + score, 0);
console.log("메서드 :", total2);
```



결과는 같음 — 339

for — i · if · += 를 직접
메서드 — 고르기 · 바꾸기 · 모으기
다시 담는 변수가 없음 — const

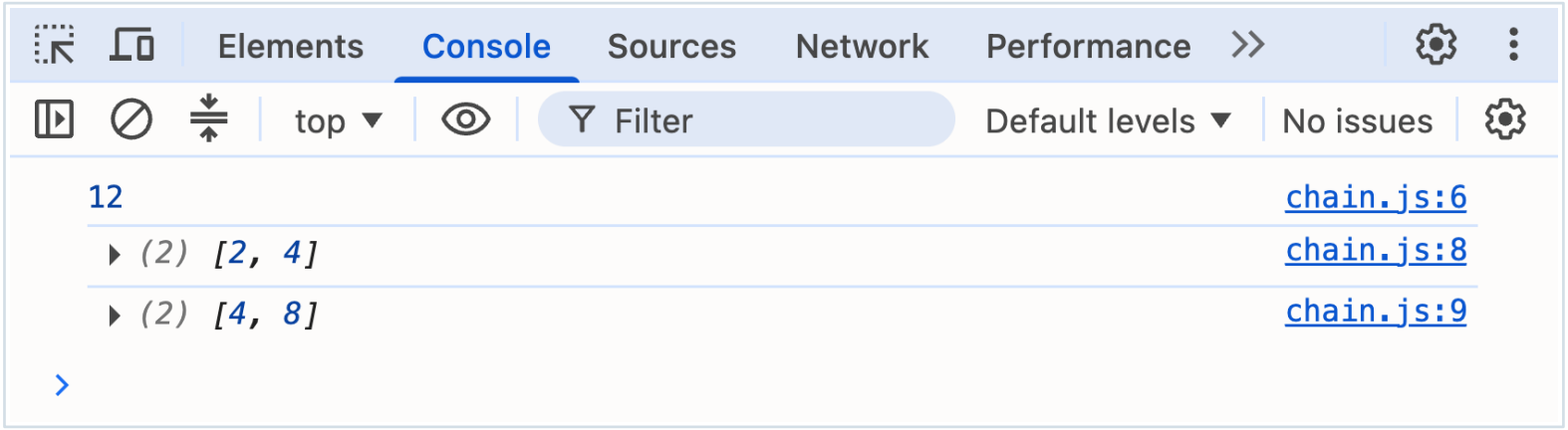
이어 쓰기 — 앞 메서드의 결과에 다음 메서드

```
const numbers = [1, 2, 3, 4, 5];
const result = numbers
  .filter((num) => num % 2 === 0)
  .map((num) => num * 2)
  .reduce((acc, curr) => acc + curr, 0);
console.log(result); // 12 (2*2 + 4*2)

console.log(numbers.filter((num) => num % 2 === 0));
console.log(numbers.filter((num) => num % 2 === 0).map((num) => num * 2));
```

한 단계씩 보면

filter — [2, 4]
map — [4, 8]
reduce — 4 + 8 = 12



reduce로 객체 만들기 — 종류별로 세기

```
const votes = ["봄", "가을", "여름", "가을", "봄", "가을"];

const counts = votes.reduce((acc, season) => {
  acc[season] = (acc[season] || 0) + 1;
  return acc;
}, {});

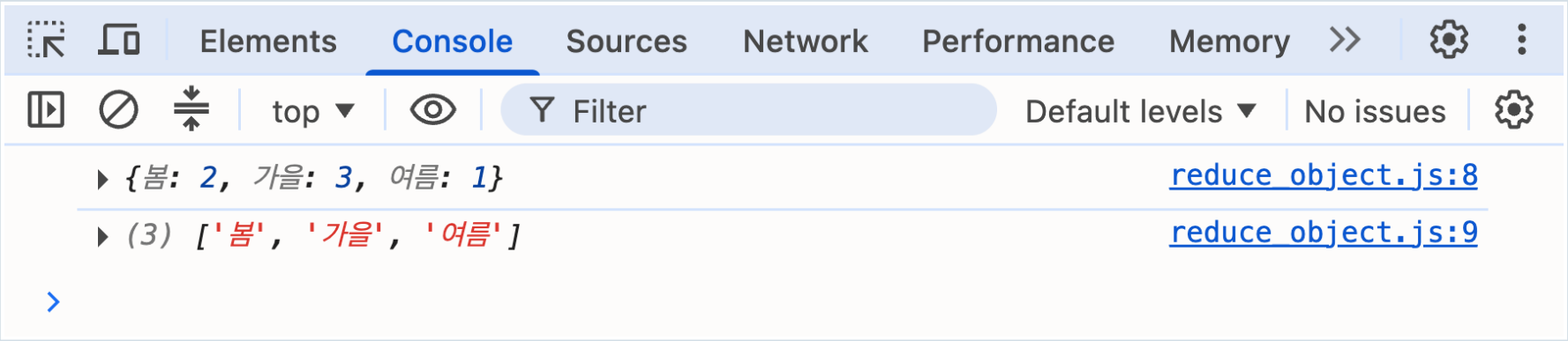
console.log(counts);
console.log(Object.keys(counts));
```

초기값 { }

acc — 지금까지 센 객체

acc[season] || 0 — 처음이면 0

return acc — 다음 차례로 넘김



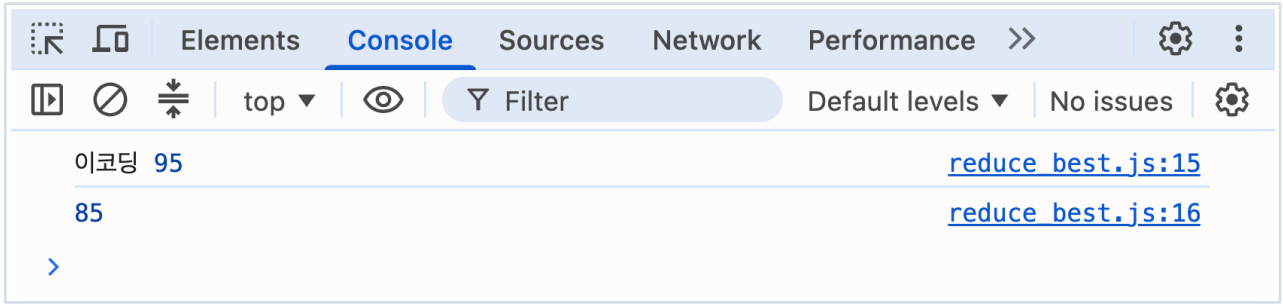
reduce로 가장 큰 것 고르기와 평균

```
const students = [
  { name: "김오즈", score: 88 },
  { name: "이코딩", score: 95 },
  { name: "박자바", score: 72 },
];

const winner = students.reduce((best, student) => {
  return student.score > best.score ? student : best;
});

const sum = students.reduce((acc, student) => {
  return acc + student.score;
}, 0);

console.log(winner.name, winner.score);
console.log(sum / students.length);
```



초기값 없이 고르기

첫 학생이 best로 시작
더 크면 바꿔 남김 — 이코딩
합계 255 ÷ 3명 = 85

04 · 고르는 법

메서드가 늘 정답은 아님

메서드가 읽기 좋을 때

고르기 · 바꾸기 · 모으기

한 단계에 한 가지 일
이름(filter · map)이 뜻을 말함
원래 배열을 바꾸지 않음

for가 읽기 좋을 때

중간에 멈추기

조건을 만나면 break
forEach · map은 멈출 수 없음
— DAY 14 11장

고르는 기준

읽는 사람이 바로 아는 쪽

reduce가 길어지면 나눠 쓰기
중간 결과에 이름 붙이기
결과가 같은지 Console로

MDN — reduce 같은 다용도 고차 함수는 강력하지만 이해하기 어려울 수 있습니다

다른 메서드나 for로 더 분명해지면, 읽기 쉬운 쪽을 고릅니다.

실습 02

한 주 운동 기록 통계

stats_loop.js를 stats.js로 복사한 뒤, 1~5번 묶음의 반복문을 하나씩 map · filter · reduce로 바꿉니다.
한 묶음을 바꿀 때마다 저장하고, 화면이 처음과 같은지 확인합니다.

만들 것 — 반복문을 메서드로

① 총 운동 시간 — reduce로 더하기
② 하루 평균 — 총합 ÷ 날 수 (그대로)
③ 가장 오래 한 날 — 초기값 없는 reduce
④ 30분 이상 한 날 — filter 뒤 map
⑤ 종류별 시간 — reduce로 객체
⑥ 화면에 쓰기 — 두 파일이 같음



반복문으로도, 메서드로도 같은 화면

실습 02

일부러 틀려 보기

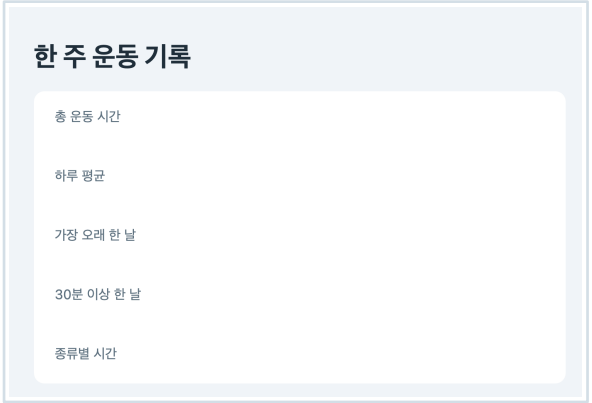
완성한 stats.js를 한 곳씩 바꿔 저장하고, 결과가 어떻게 바뀌는지 본 뒤 되돌립니다.
②만 Console에 빨간 줄이 뜹니다. 나머지는 화면을 보고 찾습니다.

바꿀 곳	결과
① 13번 줄 reduce의 , 0 지우기	총 운동 시간 「[object Object]255030358020분」, 평균 「NaN분」
② 31번 줄 return acc; 앞에 // 붙이기	숫자가 하나도 나오지 않음 — Console에 TypeError
③ 25번 줄의 >= 30을 > 30으로	30분 이상 한 날에서 목요일(30분)이 빠짐 — 4개
④ 26번 줄 .map 앞에 // 붙이기	요일 자리에 [object Object]가 다섯 개

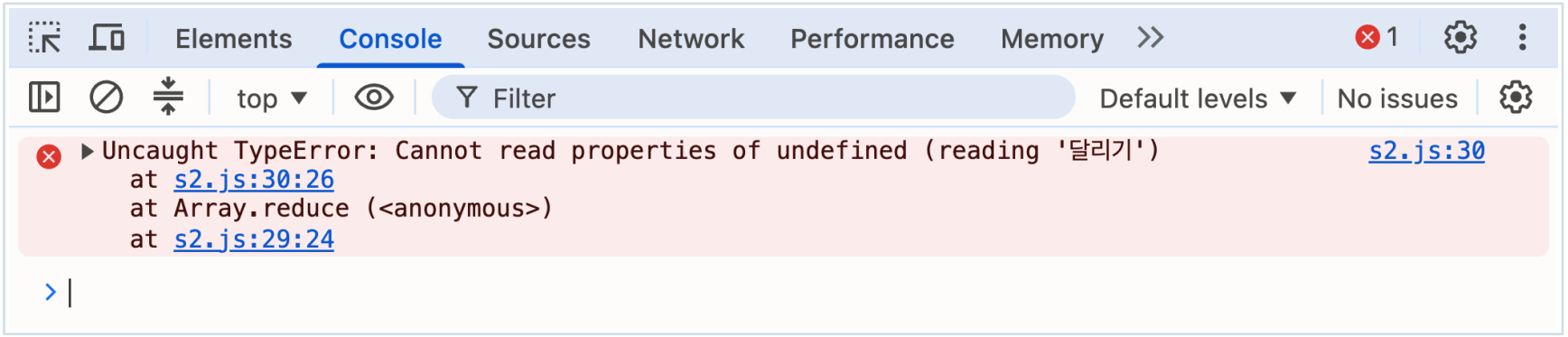
네 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 02 끝

return acc를 빠뜨리면 — 빈 화면과 TypeError

```
// stats.js 28~32번 줄 (일부러 틀린 곳 ②)
// 5. 종류별 시간 — reduce로 객체 만들기
const byType = records.reduce((acc, record) => {
  acc[record.type] = (acc[record.type] || 0) + record.minutes;
  // return acc;
}, {});
```



화면 — 숫자 칸이 모두 비어 있음



Console — reading 뒤의 「달리기」는 둘째 기록의 종류 : 둘째 차례에서 acc가 undefined

점검

고급 함수에서 자주 나오는 세 가지

!

두 카운터가 함께 움직여요

세는 변수가 함수 밖(전역)에
있으면 하나를 함께 씁니다.
만드는 함수 안으로 옮깁니다.

!

this를 잃었어요

메서드를 떼어 넘기지 않았나,
메서드를 화살표로 만들지 않았나.
화살표 콜백 안에서 불러 풉니다.

!

reduce 결과가 이상해요

초기값(0 · { })이 있는지,
함수가 acc를 return하는지
봅니다.

'어디서 만들어, 어떻게 불렀나'를 따라가면 풀립니다

클로저는 만든 곳, this는 부른 모양, reduce는 돌려준 값을 봅니다.

WRAP UP

오늘 정리

개념

스코프 — 변수가 보이는 범위

렉시컬 스코프 — 만든 곳이 바깥을 정함

클로저 — 함수 + 만들어진 곳의 변수

this — 부른 모양이 정함

Math · Date — 미리 만들어 둔 기본 객체

고차 함수 — 함수를 받거나 돌려줌

직접 쓸 수 있어야 하는 것

클로저로 값을 숨긴 카운터 만들기

만드는 함수로 따로 세는 카운터 여럿

메서드의 this와 화살표 함수로 넘기기

Math.round · Math.random · new Date()

filter → map → reduce로 이어 쓰기

reduce로 합계 · 최댓값 · 객체 만들기

참고교재

교재에서 함께 읽을 곳

오늘 수업과 이어지는 장입니다. 복습할 때 장 번호로 찾아 읽습니다.

길벗캠퍼스

『백엔드 입문자를 위한 모던 자바스크립트 & Node.js』

이창현 지음

9장. ES2015+ 문법

위키북스

『모던 자바스크립트 Deep Dive』

이웅모 지음

24장. 클로저
34장. 이터러블
26장. ES6 함수의 추가 기능
29장. Math · 30장. Date

NEXT

DAY 20

비동기 프로그래밍 심화와 브라우저 저장소

fetch 응답과 로딩 · 오류 · 성공 상태를 나눠 다루고,
localStorage로 새로고침 뒤에도 데이터를 남깁니다.

수고하셨습니다.